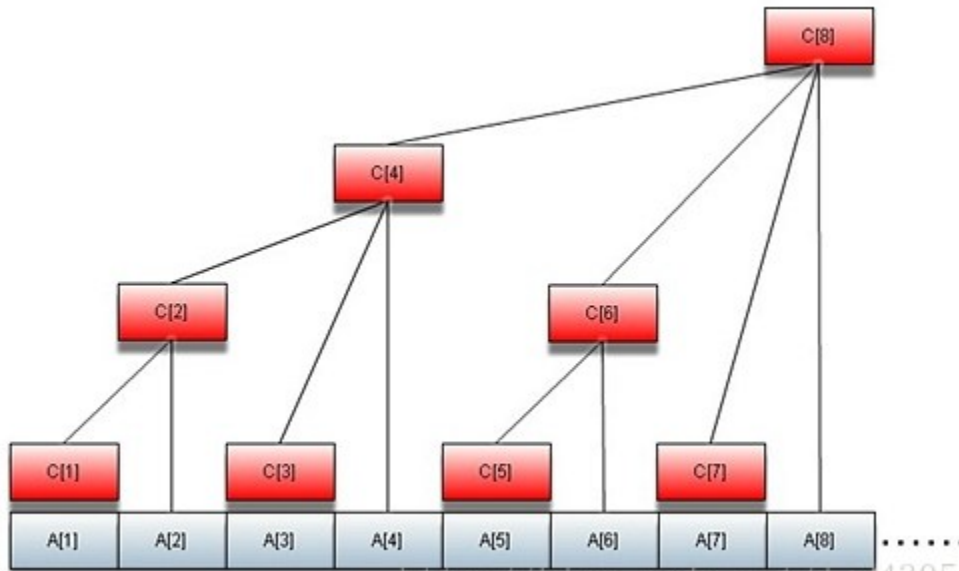


树状数组



$C[1]=A[1]$
 $C[2]=A[1]+A[2]$
 $C[3]=A[3];$
 $C[4]=A[1]+\cdots A[4];$
 $C[5]=A[5];$
 $C[6]=A[5]+A[6];$
 $C[7]=A[7];$
 $C[8]=A[1]+\cdots A[8];$

树状数组原理

写成二进制形式

$$C[0001]=A[1]$$

$$C[0010]=A[1]+A[2]$$

$$C[0011]=A[3];$$

$$C[0100]=A[1]+\cdots A[4];$$

$$C[0101]=A[5];$$

$$C[0110]=A[5]+A[6];$$

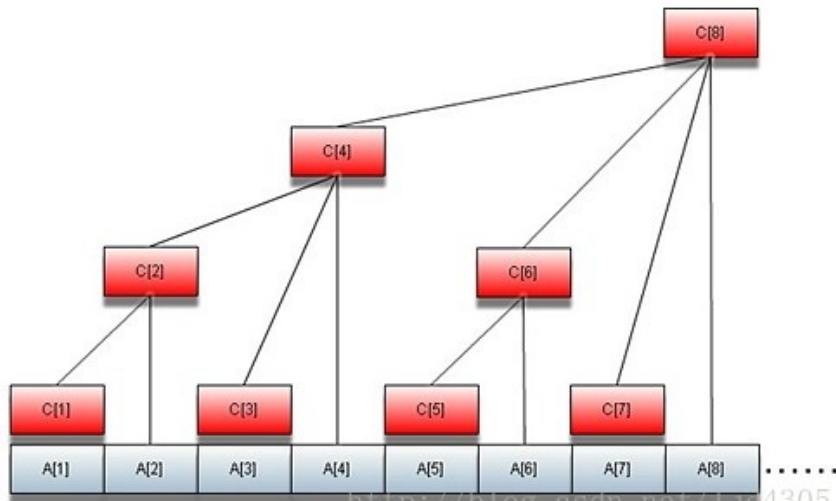
$$C[0111]=A[7];$$

$$C[1000]=A[8]+\cdots a[1];$$

规律：把十进制数转换成对应的二进制数，如果末尾是1，则 $C[i]$ 只包含它本身，如果末尾有一个0，则包含 $C[i], C[i-1]$ 。如果有两个0，则包含 $C[i]-C[i-3]$ ，如果有3个0，则包含 $C[i]-C[i-7]$ ，以此类推。

树状数组原理

所以，在计算的时候，如果我们要求前*i*个数之和。就可以根据*i*的大小以及C[k]的值来确定。



$\text{sum}[0001] = C[1];$
 $\text{sum}[0010] = C[2];$
 $\text{Sum}[0011] = C[3] + C[2];$
 $\text{Sum}[0100] = C[4];$
 $\text{Sum}[0101] = C[5] + C[4];$
 $\text{Sum}[0110] = C[6] + C[4];$
 $\text{Sum}[0111] = C[7] + C[6] + C[4];$
 $\text{Sum}[1000] = C[8]$

规律：

$$\text{sum}[0111] = \text{sum}[0110] + C[0111]$$

$$\text{Sum}[0110] = \text{sum}[0100] + C[0110]$$

$$\text{Sum}[0100] = C[0100]$$

和最后一个1的位置有关，最后一个1在第*i*位，就减去 2^{i-1} 。

位运算

虽然我们知道了怎么通过C数组求出sum的值，但是分别找出i对应的C[i]之和比较麻烦。我们都习惯了使用二进制，所以先把十进制转换成二进制，再计算，再转换成十进制特别麻烦。实际上我们可以直接通过二进制来转换

找出二进制数末尾0的个数的办法

$$X \& (-X)$$

X: 二进制数的原码

$\sim X$: 二进制数的反码(原码每位取反)

$-X$: 二进制数的补码(反码+1)

&: 与运算，同1为1，否则为0

|: 或运算，有1为1，否则为0

位运算

比如：7

X : 00000111

$\sim X$: 11111000

$-X$: 11111001

$X \& (-X)$: 00000001

7-1=6

X : 00000110

$-X$: 00000010

$X \& (-X)$: 00000010

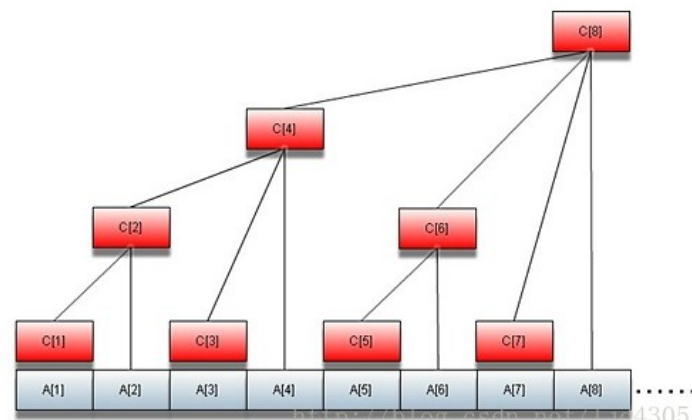
6-2=4

X : 00000100

$-X$: 00000100

$X \& (-X)$: 00000100

4-4=0



$$\text{sum}[7] = C[7] + C[6] + C[4]$$

完全符合

位运算

$X \& (-X)$: 为什么可以求末尾0的个数

设二进制数为 a b c d

那么 $\sim x$ 为 $1-a$ $1-b$ $1-c$ $1-d$

如果 $d=1$

$1-d=0$, $+1=1$

$-x$ 为 $1-a$ $1-b$ $1-c$ d

$\&$ 的结果就为1.

如果 $d=0$, 并且 $c=1$ ($ab10$)

$-x$ 为 $1-a$ $1-b$ c d

$\&$ 的结果就是10.

同理, 如果 $c=0 \dots$

建树状数组

```
void updata(int k, int x) //第k个数增加x
{
```

```
    for(int i=k; i<=n; i+=i&(-i))
    {
```

```
        tree[i]+=x;
```

```
    }
```

```
}
```

```
for(int i=1; i<=n; i++)
```

```
{
```

```
    scanf("%d", &a);
```

```
    updata(i, a); //把第i个数增加a(原来为0)
```

```
}
```

比如 $i=1$

$1+x\&(-x)=2$

$2+x\&(-x)=4$

$4+x\&(-x)=8$

这条链上的数刚好为1, 2, 4, 8

区间求和

更新的时候是正着更新，查询的时候反着计算就可以了

```
long long getsum(int x)
{
    long long sum=0;
    for(int i=x;i>0;i-=lowbit(i))
    {
        sum+=tree[i];
    }
    return sum;
}
```


三元组

题目描述

[展开](#)

Erwin最近对一种叫"thair"的东西巨感兴趣。。。

在含有 n 个整数的序列 $a_1, a_2, \dots, a_n$ 中,

三个数被称作"thair"当且仅当 $i < j < k$ 且 $a_i < a_j < a_k$

求一个序列中"thair"的个数。

输入格式

开始一个正整数 n ,

以后 n 个数 $a_1 \sim a_n$ 。

输出格式

"thair"的个数

输入输出样例

输入 #1

[复制](#)

```
4
2 1 3 4
```

输出 #1

[复制](#)

```
2
```

输入 #2

[复制](#)

```
5
1 2 2 3 4
```

输出 #2

[复制](#)

```
7
```

三元组

两个数组成的顺序对，可以直接用树状数组求解，那么三个数组成的顺序对或者逆序对呢？

如果我们还是按照前面的做法，很麻烦，所以不如直接枚举三元子序列中间那个数，然后再去找他前面出现过多少比他小的数，再去找他后面出现过多少比他大的数。

找后面出现过比他大的数很麻烦，所以我们不如反过来，倒着枚举，继续找他前面比他小的数。最后再把得到的两个答案乘起来，累加就是答案。

序列问题

【问题描述】

Tom 的梦想是成为世界冠军。因此他每天都在认真做题。某天他在学数学的时候又造出了一个新题，并准备以此为难你一下。

给定长度为 n 的序列，正整数 k ，可以定义两个函数：

$$F(i) = i^{a[i]} \% k$$

$$G(i) = a[i]^i \% k$$

现在要求计算 (l, r) 数对的数量，其中 $1 \leq l < r \leq n$ ，且 $f(l) > g(r)$ 。

【输入格式】

从文件 Calc.in 中读入数据。

第 1 行 2 个整数 n, k 。

第 2 行读入 n 个整数 $A_1, A_2, \dots, A_n$ 。

【输出格式】

输出到文件 Calc.out 中。

一行输出答案。

【样例输入】

5 10000

3 1 5 4 2

【样例输出】

2

序列问题

题目中要求 $l < r$ 并且 $f[l] > g[r]$ ，妥妥的求解逆序对的题，先预处理出来所有的 $f[i]$ 和 $g[i]$ 。我们以前求解的逆序对都是在同一个数组中求解，只是该题是一个二元组，不过原理都是一样的，比如我以 $f[i]$ 来建立树状数组，每次先拿 $g[i]$ 进去找有多少个 $f[i]$ 比他小的，然后再把当前的 $f[i]$ 放进去。

注意该题数组比较大，需要离散化，但是由于是两个数组，所以需要放到一个数组里进行离散化，不然离散化标准不一样，答案就是错误的。

比如 1 2 3 1 4 6

单独离散化都是 1 2 3，直接比较答案肯定错了

区间修改区间查询

树状数组的区间修改区间查询稍微复杂一点，其实也比较简单，用习惯了就好，他还是基于差分的思想。

设原数组第 i 位的值为 a_i ， $d_i = a_i - a_{i-1}$ ，则有(这里认为 $a_0 = 0$)：

$$a_x = \sum_{i=1}^x d_i$$

所以有：

$$\sum_{i=1}^x a_i = \sum_{i=1}^x \sum_{j=1}^i d_j = \sum_{i=1}^x (x - i + 1) d_i$$

于是我们得到了：

$$\sum_{i=1}^x a_i = (x + 1) \sum_{i=1}^x d_i - \sum_{i=1}^x d_i \times i$$

区间修改 区间查询

看不懂，没关系，我们来分析一下，设原数组为 $a[i]$ ，差分数组为 $d[i]$ ，前缀和数组为 $c[i]$ 。那么就有

$$a[1]=d[1];$$

$$a[2]=d[1]+d[2];$$

$$a[3]=d[1]+d[2]+d[3];$$

...

$$a[n]=d[1]+d[2]+\cdots+d[n];$$

我们来观察一下 c 数组和 d 数组的关系

对于 $c[i]$ 来说， $d[1]$ 被加了 i 次， $d[2]$ 被加了 $i-1$ 次， $d[i]$ 被加了1次。再根据乘法分配律就得到了：

$$\sum_{i=1}^x a_i = (x+1) \sum_{i=1}^x d_i - \sum_{i=1}^x d_i \times i$$

区间修改 区间查询

所以对于区间修改区间查询的问题，我们只需要维护两个树状数组就可以了，一个维护差分数组 $d[i]$ ，另一个维护 $d[i]*i$ 的值。

```
void updata(int k,int x)
{
    for(int i=k;i<=n;i+=lowbit(i))
    {
        t1[i]+=x;  t2[i]+=(long long)x*k;
    }
}
long long query(int k)
{
    long long ans=0;
    for(int i=k;i>0;i-=lowbit(i))
        ans+=(k+1)*t1[i]-t2[i];
    return ans;
}
```

二维树状数组

树状数组求解的问题必须要有区间可加可减性，否则就很难甚至不可以用树状数组求解，比如常见的求最大最小值，因为不具有区间可减性，所以不可以用树状数组求解，树状数组求得的都是前缀答案。

树状数组也有他的优势，就可以能够很快的拓展到二维甚至多维。就拿二维树状数组来说，也就是单点修改，平面查询，可以很快的用二维树状数组解决，如果是二维线段树，不但实现很复杂，而且代码长度翻倍。

而二维树状数组实现很简单，只需要增加一行代码。

二维树状数组

```
void updata(int x, int y, int z)
{
    for(int i=x; i<=n; i+=lowbit(i))
        //在一维的枚举上加一维，三维也是一样
        for(int j=y; j<=n; j+=lowbit(j))
            b[i][j]+=z;
}

long long query(int x, int y)
{
    long long sum=0;
    for(int i=x; i>0; i-=lowbit(i))
        for(int j=y; j>0; j-=lowbit(j))
            sum+=b[i][j];
    return sum;
}
```

线段树的建树

```
void build(int id,int l,int r)
{
    tree[id].l=l;
    tree[id].r=r;
    if(l==r)//递归终止条件
    {
        tree[id].max=a[l];
        tree[id].sum=a[l];
        return ;
    }
    int mid=(l+r)/2;
    build(id*2,l,mid);//编号为id的节点的儿子是固定的2*id和2*id+1
    build(id*2+1,mid+1,r);
    tree[id].max=max(tree[id*2].max,tree[id*2+1].max);
    tree[id].sum=tree[id*2].sum+tree[id*2+1].sum;
    //递归回去的时候更新父亲节点的max和sum值
    return ;
}
```

单点更新

```
void update(int id,int k,int x)
{
    if(tree[id].l==tree[id].r)
    {
        tree[id].max=x;
        tree[id].sum=x;
        return ;
    }
    int mid=(tree[id].l+tree[id].r)/2;
    if(k<=mid)//在左区间就更新左区间
        update(id*2,k,x);
    else//在右区间就更新右区间
        update(id*2+1,k,x);
    tree[id].max=max(tree[id*2].max,tree[id*2+1].max);
    tree[id].sum=tree[id*2].sum+tree[id*2+1].sum;
    //子节点信息被修改，其父亲节点信息也要更新
}
```

区间查询

```
void query(int id,int l,int r)
{
    if(tree[id].l==l&&tree[id].r==r)// 找到一个完整区间
    {
        return tree[id].sum;
    }
    int mid=(tree[id].l+tree[id].r)/2;
    if(r<=mid)// 都在左区间
        return query(id*2,l,r);
    else if(l>mid)// 都在右区间
        return query(id*2+1,l,r);
    else// 左右子树都有
        return query(id*2,l,mid)+query(id*2+1,mid+1,r);
}
```

区间更新

```
void update(int id,int l,int r,int x)
{
    if(tree[id].l==l&&tree[id].r==r)
    {
        // 修改懒标记
        // 计算区间真实和
        return ;
    }
    // 如果在下一层, 先标记下传, 再遍历儿子
    if(tree[id].lazy)
        pushdown(id);
    int mid=(tree[id].l+tree[id].r)/2;
    if(r<=mid)// 和区间查询几乎一致
        update(id*2,l,r,x);
    else
        update(id*2+1,l,r,x);
    else
    {
        update(id*2,l,mid);
        update(id*2+1,mid+1,r);
    }
    tree[id].sum=tree[id*2].sum+tree[id*2+1].sum;
}
```

懒标记下传

```
void down(int id)
{
    tree[id*2].lazy+=tree[id].lazy;
    tree[id*2+1].lazy+=tree[id].lazy; // 先下传父亲节点懒标记
    tree[id*2].sum+=(tree[id*2].r-tree[id*2].l+1)*tree[id].lazy;
    tree[id*2+1].sum+=(tree[id*2+1].r-tree[id*2+1].l+1)*tree[id].lazy;
    // 同时还要更新区间之和，注意是*新加的懒标记而不是总共的懒标记
    tree[id].lazy=0; // 懒标记清零
}
```

区间更新时要注意懒标记下传的位置，不能再区间判等之前，因为这样会导致访问到叶子节点的下下一个节点，所以会导致RE，如果放在更新左右区间之后肯定就错误了。

区间查询

```
void query(int id,int l,int r)
{
    if(tree[id].l==l&&tree[id].r==r)
    {
        return tree[id].sum;
    }
    if(tree[id].lazy)
        pushdown(id);
    int mid=(l+r)/2;
    if(r<=mid)
        return query(id*2,l,r);
    else
        return query(id*2+1,l,r);
    else
        return query(id*2+1,l,mid)+query(id*2+1,mid+1,r);
}
```

贴海报

题目描述

Bytetown城市要进行市长竞选，所有的选民可以畅所欲言地对竞选市长的候选人发表言论。为了统一管理，城市委员会为选民准备了一个张贴海报的electoral墙。张贴规则如下：

1. electoral墙是一个长度为N个单位的长方形，每个单位记为一个格子；
2. 所有张贴的海报的高度必须与electoral墙的高度一致的；
3. 每张海报以“A B”表示，即从第A个格子到第B个格子张贴海报；
4. 后贴的海报可以覆盖前面已贴的海报或部分海报。

现在请你判断，张贴完所有海报后，在electoral墙上还可以看见多少张海报（只要一张海报有部分被看见就算被看见）。

贴海报

该题就是一道线段树的简单题，对于线段树的题，你只需要想到如果我知道了左右两个区间的信息，如何维护父亲结点的信息就可以了。

如果我们知道了两个子区间的能看见的海报种类，要怎么求合并之后的区间海报种类呢？我们还需要判断相邻的端点处是否相同，如果相同，那么答案就是之和-1，如果不相同，就是之和，所以我们除了维护海报种类之外，还需要维护区间左右端点信息。

该题因为数据范围比较大，所以需要离散化，但是直接把区间左右端点离散化是错误的。

贴海报

比如 $[5, 7]$, $[1, 5]$, $[7, 8]$

离散化之后是 $[2, 3]$, $[1, 2]$, $[3, 4]$

离散化之后的结果是第一张海报被覆盖了，那么能看见的海报只有两张，但是实际上并不是第一章海报并没有完全覆盖，中间还有空白区域。

那么对于这种情况，如果我们直接离散化成 $1, 3, 5 \dots$ 是错误的，因为如果区间长度本来就只有2，那么我们相当于把区间增大了。我们就在中间加上一个点，如果一个区间长度大于1，那么离散化的时候就多加入进去一个中间的点，比如 $[5, 10]$ ，可以随便加一个中间的点6—9，但是如果是 $[5, 6]$ 就不能加点了，因为本来区间长度就只有2。

方差

题目描述

蒟蒻 HansBug 在一本数学书里面发现了一个神奇的数列，包含 N 个实数。他想算算这个数列的平均数和方差。

输入格式

第一行包含两个正整数 N, M ，分别表示数列中实数的个数和操作的个数。

第二行包含 N 个实数，其中第 i 个实数表示数列的第 i 项。

接下来 M 行，每行为一条操作，格式为以下三种之一：

操作 1：1 x y k ，表示将第 x 到第 y 项每项加上 k ， k 为一实数。

操作 2：2 x y ，表示求出第 x 到第 y 项这一子数列的平均数。

操作 3：3 x y ，表示求出第 x 到第 y 项这一子数列的方差。

方差

操作有三个，第一个是区间加上一个值，这么很好维护；第二个是区间平均值，直接维护一个区间之和，计算的时候除以区间长度就可以了；关键是第三个操作，求解区间方差，首先我们要知道方差的计算公式：

$$(x_1-x)^2+(x_2-x)^2+\cdots(x_n-x)^2$$

当区间不固定的时候，区间的平均值 x 也不固定，所以我们没法维护 x_i-x 。

我们可以考虑把括号拆开，那么表达式就变为了：

$$x_1^2+x_2^2+\cdots x_n^2-2x(x_1+x_2+\cdots x_n)+n*x^2$$

所以我们只需要再维护一个平方和就可以了，剩下的部分只需要算出总和和平均值，实际上就是算和

无聊的数列

维护一个数列 a_i ，支持两种操作：

- `1 l r K D`：给出一个长度等于 $r - l + 1$ 的等差数列，首项为 K ，公差为 D ，并将它对应加到 $[l, r]$ 范围中的每一个数上。即：令 $a_l = a_l + K, a_{l+1} = a_{l+1} + K + D \dots a_r = a_r + K + (r - l) \times D$ 。
- `2 p`：询问序列的第 p 个数的值 a_p 。

输入格式

第一行两个整数数 n, m 表示数列长度和操作个数。

第二行 n 个整数，第 i 个数表示 a_i 。

接下来的 m 行，每行先输入一个整数 opt 。

若 $opt = 1$ 则再输入四个整数 $l\ r\ K\ D$ ；

若 $opt = 2$ 则再输入一个整数 p 。

无聊的数列

线段树的区间修改只能在区间上修改或者变化为一个值，所以是不能直接区间修改一个等差数列的，但是该题的等差数列怎么办呢？

等差数列要怎么才可以变化为区间一个相等的序列，那就是差分，并且该题刚好最后是单点询问

所以我们先把数组差分，区间加上一个等差数列就变成了区间除左端点外增加一个公差，左端点增加一个首先，右端点的下一个位置减少一个首项。

那么接下来就是裸的线段树区间修改区间查询了，最后前缀查询就得到答案。

动态最值

有一个包含 n 个元素的数组，要求实现以下操作：

- `DELETE k`：删除位置 k 上的数。右边的数往左移一个位置。
- `QUERY i j`：查询位置 $i \sim j$ 上所有数的最小值和最大值。

例如有 10 个元素：

位置	1	2	3	4	5	6	7	8	9	10
元素	1	5	2	6	7	4	9	3	1	5

`QUERY 2 8` 的结果为 `2 9`。依次执行 `DELETE 3` 和 `DELETE 6`（注意这时删除的是原始数组的元素 7）后数组变为：

位置	1	2	3	4	5	6	7	8
元素	1	5	6	7	4	3	1	5

`QUERY 2 8` 的结果为 `1 7`。

动态最值

一般情况下，线段树是没有办法做涉及点删除和增加操作的题的，因为线段树的区间是最开始就预处理出来的，一旦涉及到点的删除和增加，那么所有预处理出来的区间都是错的，都需要重新初始化，那么效率肯定就过不去了。

但是对于一些特殊的修改，比如只有删除没有添加，或者每次在固定位置添加这一类操作，实际上线段树还可以可以做的，就是当做平衡树来处理。

对于这种点删除的情况，我们可以当做区间不动，未删除的点是真儿子，删除的点是假儿子，那么我查询区间端点的时候，不再是通过 l, r, mid 来查询，而是通过新开的变量 son 来查询， son 表示当前区间真实儿子数量。



动态最值

第一步：线段树建树

和常规线段树建树没区别，只是多记录一个区间真实节点数量，最开始的时候所有节点都存在，所以节点数量就是区间长度。



动态最值

第二步：线段树单点删除

假设我要求删除当前序列中第 k 个节点，根当前节点开始，不再是和 mid 做比较，因为 mid 和区间真实节点数量没有任何关系，我们判断在左区间还是在右区间，那么比较的就是左右区间真实节点数量，所以用 k 和 $tree[id*2].son$ 做比较，如果 $k < tree[id*2].son$ ，说明查找的位置在左区间，否则查找的位置在右区间。但是我们要注意那么我们应该在右区间查找第几个节点呢？还是第 k 个？并不是，因为右子树的节点是不包含左子树的，所以应该在右子树里面查找第 $k - tree[id*2].son$ 个节点。这是最容易错的地方， son 只是表示当前节点的儿子数量，也就是没有算左子树的儿子个数，所以要减去。

动态最值

```
void update(int id,int k)
{
    if(tree[id].l==tree[id].r)//既然是单点修改, 所以是修改叶子节点
    {
        tree[id].son=0;//真实儿子个数为0
        tree[id].Max=-inf;//存在负数
        tree[id].Min=inf;
        return ;
    }
    if(k<=tree[id*2].son)//和当前节点左儿子的节点数作比较
    update(id*2,k);
    else
    update(id*2+1,k-tree[id*2].son);//注意要减去左儿子数量
    tree[id].Max=max(tree[id*2].Max,tree[id*2+1].Max);
    tree[id].Min=min(tree[id*2].Min,tree[id*2+1].Min);
    tree[id].son=tree[id*2].son+tree[id*2+1].son;
}
```

动态最值

第三步：线段树区间查询

对于当前节点，如果我要查询的区间是 $[l, r]$ ，那么如果 $r \leq \text{tree}[\text{id} * 2].\text{son}$ ，那么说明这一段都在左区间，那么去左区间查找，如果 $l > \text{tree}[\text{id} * 2].\text{son}$ ，那么我们就去右区间去查找这一段，但是右子树同样不包含左子树的节点数，所以 l 和 r 都要同时减去

$\text{tree}[\text{id} * 2].\text{son}$ ，再去右区间查找，不能直接查找区间 $[l, r]$ 。如果两个区间都有一部分，那么左区间的能查找的最后一个数就是 $\text{tree}[\text{id} * 2].\text{son}$ ，在右区间查找的第一个数就是 $\text{tree}[\text{id} * 2 + 1].\text{son}$

那么什么时候查找结束呢？ $r - l + 1 = \text{区间长度}$ 或者 $l = 1$ ， $r = \text{tree}[\text{id}].\text{son}$ 的时候，这两者只要一个满足，另一个也一定满足

线段树

```
node query(int id,int l,int r)
{
    //if(l==1&&r==tree[id].son)
    if(r-l+1==tree[id].son)//如果找到一个完整区间
        return tree[id];
    if(r<=tree[id*2].son)//都在左区间
        return query(id*2,l,r);
    else if(l>tree[id*2].son)//都在右区间
        return query(id*2+1,l-tree[id*2].son,r-tree[id*2].son);
    else
    {
        node t1=query(id*2,l,tree[id*2].son);
        node t2=query(id*2+1,1,r-tree[id*2].son);
        node tmp;
        tmp.Max=max(t1.Max,t2.Max);
        tmp.Min=min(t1.Min,t2.Min);
        return tmp;
    }
}
```

线段树

前面只是涉及到线段树单点删除的操作，区间删除的做法和单点删除是一样的，参照区间查询。

但是如果是区间增加呢？先以简单的单点增加为例，如果是随意加点，那么线段树是做不了的，比如在每次增加操作在第 i 个数之后添加一个数，后面的数后移。但是有一种特殊的增加他是可以的，比如每次都在末尾添加一位数，或者在开头添加一个数，或者在固定位置添加一个数。

那么对于这一类的题应该如何做呢？

线段树

对于删除操作我们知道怎么做了，但是对于增加操作我们怎么处理呢？实际上我们可以看做删除操作，这里介绍一种办法——留坑法，先看下有多少次增加操作，那么最开始建线段树的时候就以 $n+k$ 为区间建立线段树，而不是以 $1—n$ 为区间建树，但是最开始把区间 $[n+1, n+k]$ 的所有节点设置为失效节点。如果有插入操作，那么我就把该位置的点变为有效节点。

这就是线段树的一种失效节点，有效节点的办法，如果有添加操作，就把失效节点变为有效节点，如果有删除节点，就把有效节点变为失效节点，每次只统计有效节点的答案。

买票

题目描述

在中国的农历新年期间很难买到铁路车票，所以必须早起，加入长队……

农历新年即将来临，但不幸的是，小猫仍然要奋斗在OI的世界里。现在，他不得不乘火车前往四川绵阳，参加信息学奥林匹克的冬季训练营。

现在是凌晨一点，外面黑了。来自西北的寒风并没有吓跑排队的人。由于天气很黑，前后两个人也看不太清楚，并且场面也很混乱，所以就有一些人经常悄悄跑进来插队。

现在已知每个插队的人的插入的位置，问所有人插完队之后的序列。

买票

这道题涉及到了插入操作，但是还是可以用线段树来处理。首先，我们知道插入之后最多有 n 个人，那么我们先预留 n 为空位置（真实儿子数量都为0），如果我们顺着来做，那么如果每次插入都在1，那么很明显我们需要在每个位置都预留 n 个空位，肯定不行，所以我们反向思考，因为最后一个人插入之后，他的位置一定是给定的 k ，即最后一个人插入的位置就是线段树上第 k 个位置，那么我们再考虑倒数第二个人，如果他插入的位置在 k 前，那么他的位置也是真实位置，否则就是给定位置+1，前面的所有人都是相同思考方法，看看前面已经有多少个插入了，再去还未插入人的空位置找 k 个。



练习题

先做三道线段树求和，求极值，树状数组的水题

MSOJ	1440	三元上升子序列
MSOJ	1407	序列
MSOJ	1441	矩阵
MSOJ	1512	贴海报
洛谷	1438	无聊的数列
MSOJ	1525	动态最值
MSOJ	1528	买票
洛谷	2073	送花

